MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

Name of Faculty: MOHIT SHARMA

Subject & Code: ANALYSIS OF ALGORITHM & 6CS4-02

Branch: CSE 6TH SEM Batch: A&B

Semester: VIth **Session:** 2023-24(even Sem.)

Unit No.: 4 **Date of Submission: 09-04-2024**

Lect.No.	Topics Covered	Page No.	Total pages
1	Branch and bound	2-11	10
2	LC searching Bounding	12-14	3
3	FIFO branch and bound	15-21	7
4	LC branch and bound	22-23	2
5	Applications of branch and bound	24-25	2
6	0/1 Knapsack problem	25-29	5

References Book:

1. T .H .Cormen, C .E .Leiserson, R .L . Rivest "Introduction to Algorithms", 3rd Ed.,PHI, 2011
2. **E. Horowitz, S. Sahni, and S. Rajsekaran, "Fundamentals of Computer Algorithms,"Galgotia Publication**
3. **S.K Basu- Design Methods and Analysis of Algorithms, 2nd Ed., PHI**

FACULTY HOD DEAN ACADEMICS

UNIT 4

LECTURE 1

BRANCH AND BOUND

What is Branch and bound?

Branch and bound is one of the techniques used for problem solving. It is similar to the backtracking since it also uses the state space tree. It is used for solving the optimization problems and minimization problems. If we have given a maximization problem then we can convert it using the Branch and bound technique by simply converting the problem into a maximization problem.

Let's understand through an example.

Jobs = {j1, j2, j3, j4}

P = {10, 5, 8, 3}

d = {1, 2, 1, 2}

The above are jobs, problems and problems given. We can write the solutions in two ways which are given below:

Suppose we want to perform the jobs j1 and j2 then the solution can be represented in two ways:

The first way of representing the solutions is the subsets of jobs.

S1 = {j1, j4}

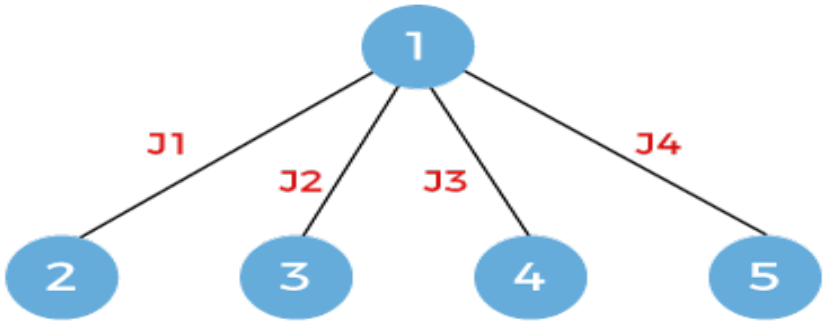
The second way of representing the solution is that first job is done, second and third jobs are not done, and fourth job is done.

S2 = {1, 0, 0, 1}

The solution s1 is the variable-size solution while the solution s2 is the fixed-size solution.

First, we will see the subset method where we will see the variable size.

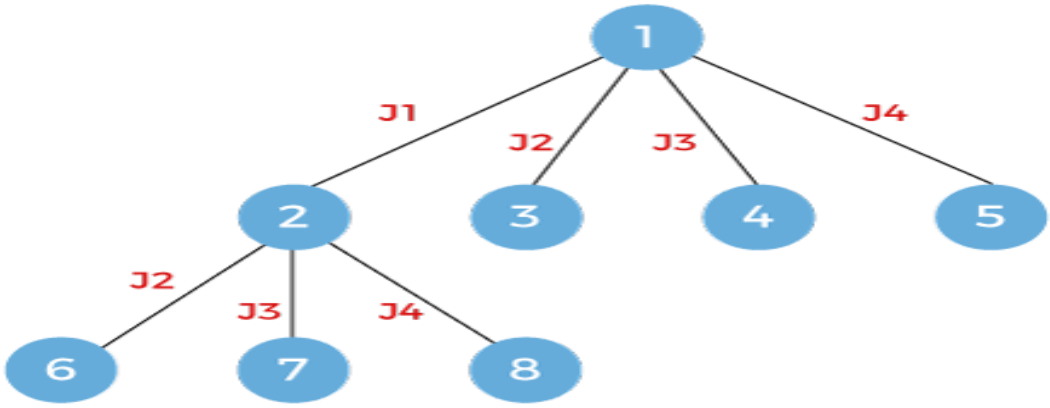
First method:



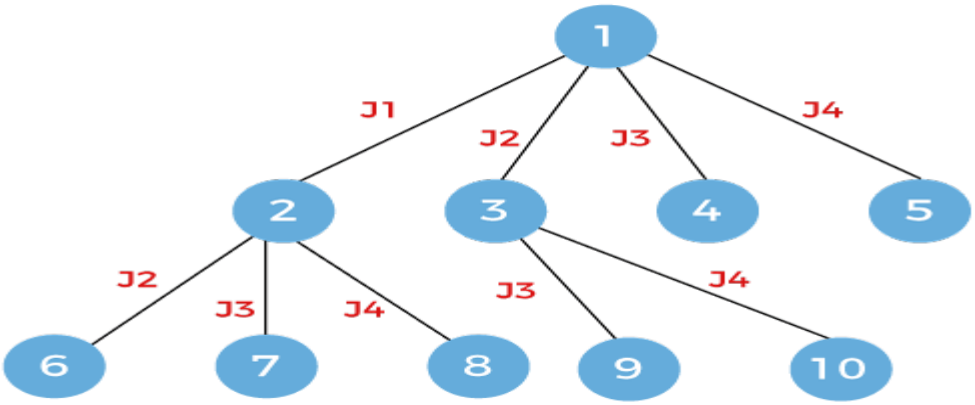
In this case, we first consider the first job, then second job, then third job and finally we consider the last job.

As we can observe in the above figure that the breadth first search is performed but not the depth first search. Here we move breadth wise for exploring the solutions. In backtracking, we go depth-wise whereas in branch and bound, we go breadth wise.

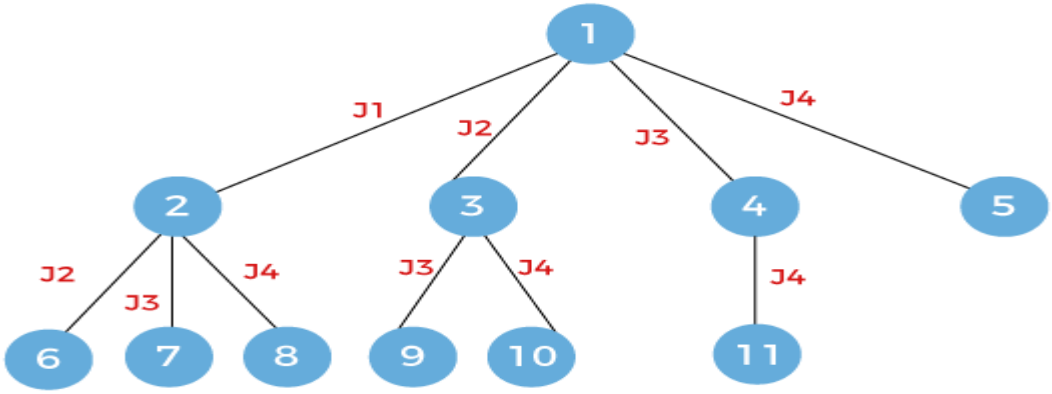
Now one level is completed. Once I take first job, then we can consider either j2, j3 or j4. If we follow the route then it says that we are doing jobs j1 and j4 so we will not consider jobs j2 and j3.



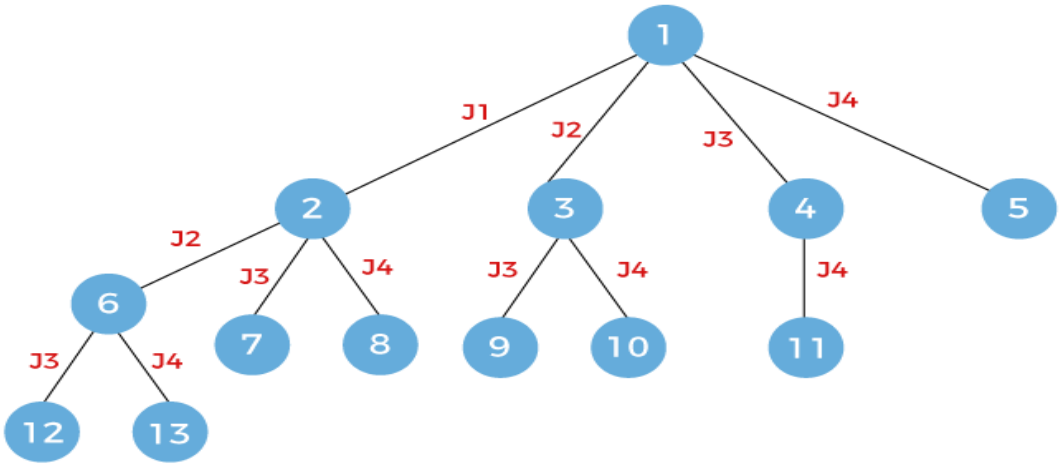
Now we will consider the node 3. In this case, we are doing job j2 so we can consider either job j3 or j4. Here, we have discarded the job j1.



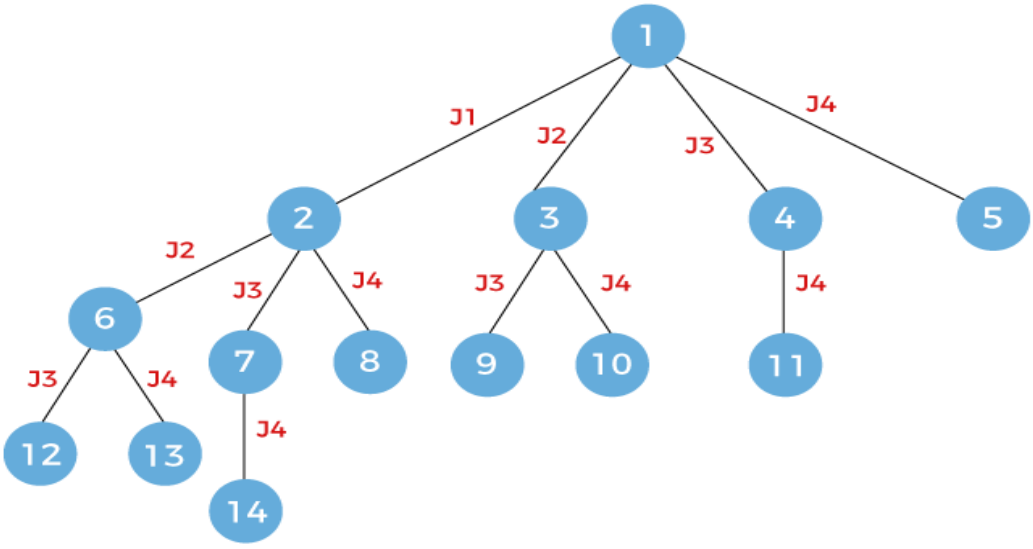
Now we will expand the node 4. Since here we are doing job j3 so we will consider only job j4.



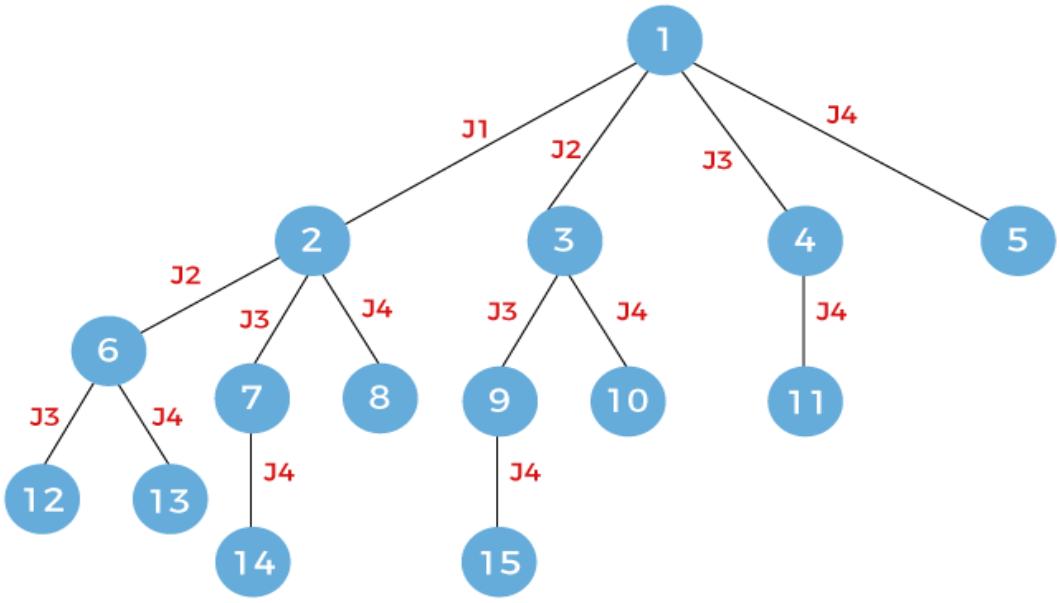
Now we will expand node 6, and here we will consider the jobs j3 and j4.



Now we will expand node 7 and here we will consider job j4.



Now we will expand node 9, and here we will consider job j4.



The last node, i.e., node 12 which is left to be expanded. Here, we consider job j4.

The above is the state space tree for the solution s1 = {j1, j4}

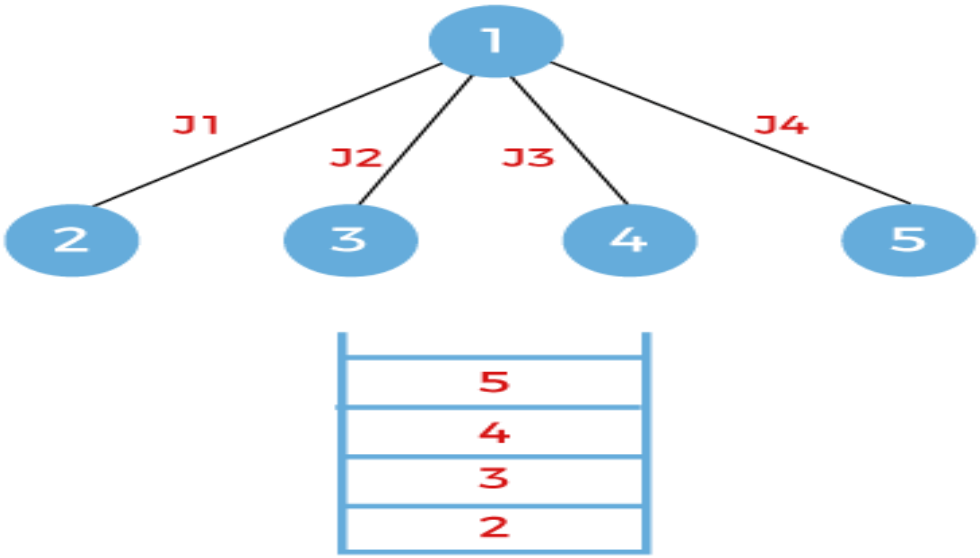
Second method:

We will see another way to solve the problem to achieve the solution s1.

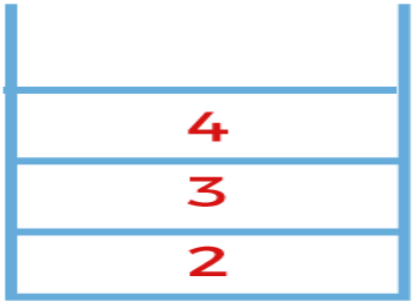
First, we consider the node 1 shown as below:

Now, we will expand the node 1. After expansion, the state space tree would be appeared as:

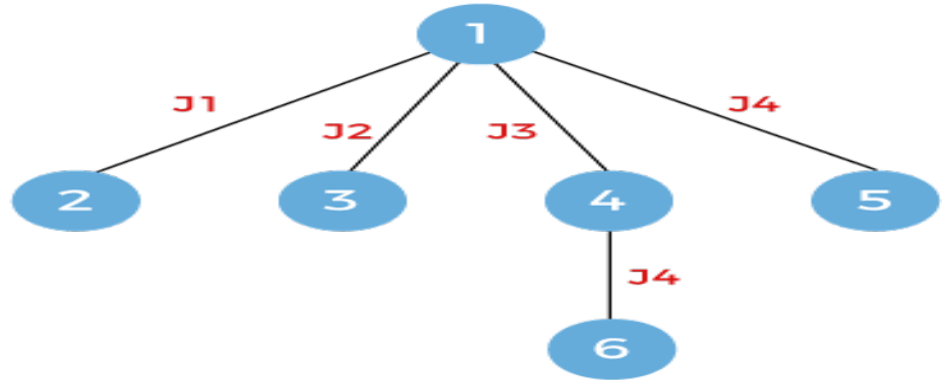
On each expansion, the node will be pushed into the stack shown as below:

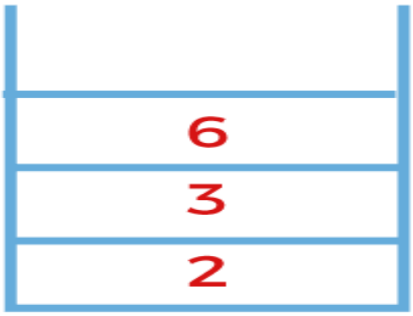


Now the expansion would be based on the node that appears on the top of the stack. Since the node 5 appears on the top of the stack, so we will expand the node 5. We will pop out the node 5 from the stack. Since the node 5 is in the last job, i.e., j4 so there is no further scope of expansion.

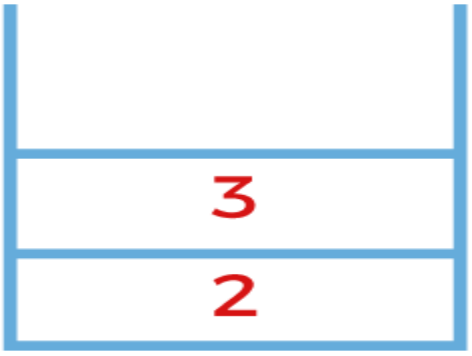


The next node that appears on the top of the stack is node 4. Pop out the node 4 and expand. On expansion, job j4 will be considered and node 6 will be added into the stack shown as below:

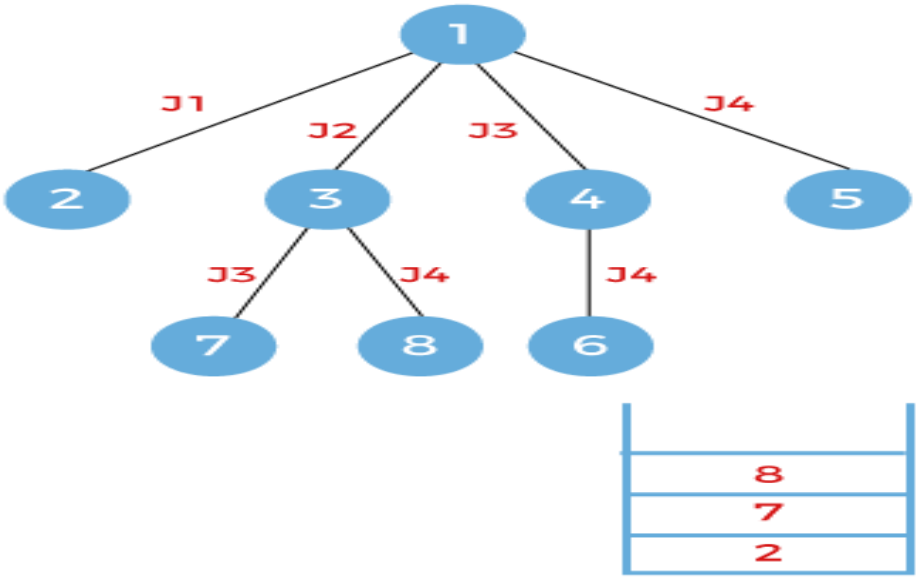




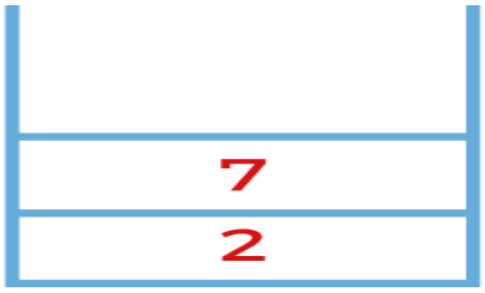
The next node is 6 which is to be expanded. Pop out the node 6 and expand. Since the node 6 is in the last job, i.e., j4 so there is no further scope of expansion.



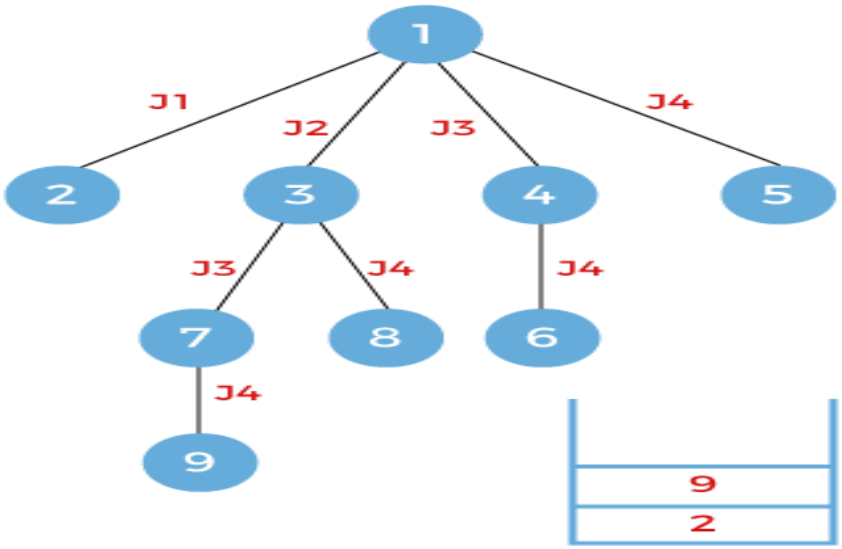
The next node to be expanded is node 3. Since the node 3 works on the job j2 so node 3 will be expanded to two nodes, i.e., 7 and 8 working on jobs 3 and 4 respectively. The nodes 7 and 8 will be pushed into the stack shown as below:



The next node that appears on the top of the stack is node 8. Pop out the node 8 and expand. Since the node 8 works on the job j4 so there is no further scope for the expansion.



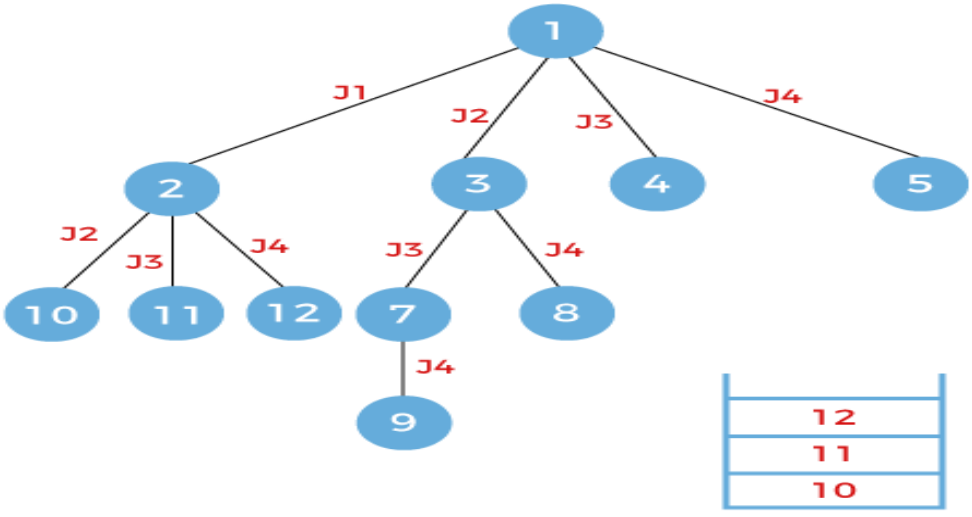
The next node that appears on the top of the stack is node 7. Pop out the node 7 and expand. Since the node 7 works on the job j3 so node 7 will be further expanded to node 9 that works on the job j4 as shown as below and the node 9 will be pushed into the stack.



The next node that appears on the top of the stack is node 9. Since the node 9 works on the job 4 so there is no further scope for the expansion.



The next node that appears on the top of the stack is node 2. Since the node 2 works on the job j1 so it means that the node 2 can be further expanded. It can be expanded upto three nodes named as 10, 11, 12 working on jobs j2, j3, and j4 respectively. There newly nodes will be pushed into the stack shown as below:



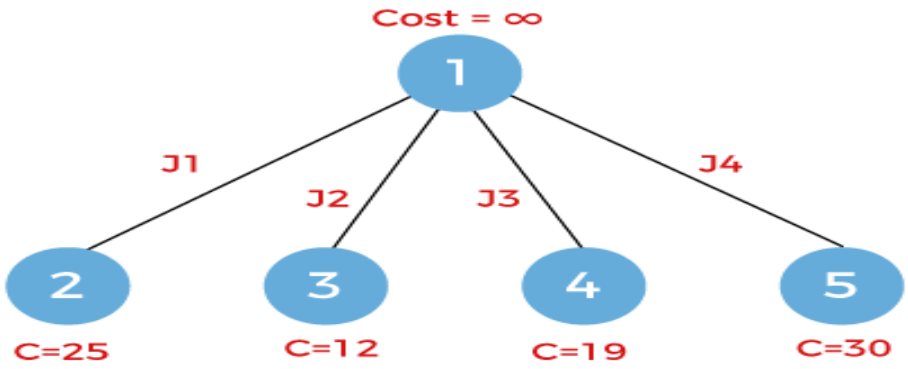
In the above method, we explored all the nodes using the stack that follows the LIFO principle.

Third method

There is one more method that can be used to find the solution and that method is Least cost branch and bound. In this technique, nodes are explored based on the cost of the node. The cost of the node can be defined using the problem and with the help of the given problem, we can define the cost function. Once the cost function is defined, we can define the cost of the node.

Let's first consider the node 1 having cost infinity shown as below:

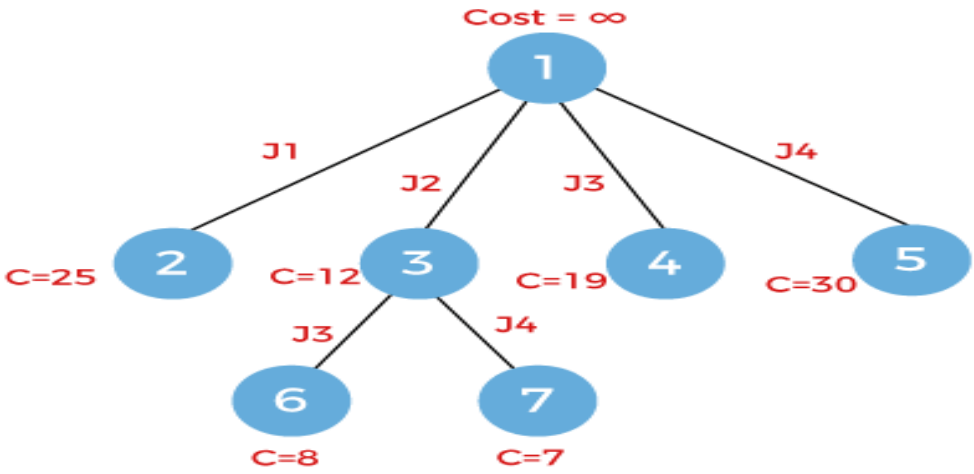
Now we will expand the node 1. The node 1 will be expanded into four nodes named as 2, 3, 4 and 5 shown as below:



Let's assume that cost of the nodes 2, 3, 4, and 5 are 25, 12, 19 and 30 respectively.

Since it is the least cost branch n bound, so we will explore the node which is having the least cost. In the above figure, we can observe that the node with a minimum cost is node 3. So, we will explore the node 3 having cost 12.

Since the node 3 works on the job j2 so it will be expanded into two nodes named as 6 and 7 shown as below:



The node 6 works on job j3 while the node 7 works on job j4. The cost of the node 6 is 8 and the cost of the node 7 is 7. Now we have to select the node which is having the minimum cost. The node 7 has the minimum cost so we will explore the node 7. Since the node 7 already works on the job j4 so there is no further scope for the expansion.

Lecture 2

There are many optimization problems in computer science, many of which have a finite number of the feasible shortest path in a graph or minimum spanning tree that can be solved in polynomial time. Typically, these problems require a worst-case scenario of all possible permutations. The branch and bound algorithm create branches and bounds for the best solution.

In this tutorial, we'll discuss the branch and bound method in detail.

Different search techniques in branch and bound:

The Branch algorithms incorporate different search techniques to traverse a state space tree. Different search techniques used in B&B are listed below:

1. LC search
2. BFS
3. DFS

1. LC search (Least Cost Search):

It uses a heuristic cost function to compute the bound values at each node. Nodes are added to the list of live nodes as soon as they get generated.

The node with the least value of a cost function selected as a next E-node.

2.BFS(Breadth First Search):

It is also known as a FIFO search.

It maintains the list of live nodes in first-in-first-out order i.e, in a queue,

The live nodes are searched in the FIFO order to make them next E-nodes.

3. DFS (Depth First Search):

It is also known as a LIFO search.

It maintains the list of live nodes in last-in-first-out order i.e. in a stack.

The live nodes are searched in the LIFO order to make them next E-nodes.

When to apply Branch and Bound Algorithm?

Branch and bound is an effective solution to some problems, which we have already discussed. We'll discuss all such cases where branching and binding are appropriate in this section.

- It is appropriate to use a branch and bound approach if the given problem is discrete optimization. Discrete optimization refers to problems in which the variables belong to the discrete set. Examples of such problems include 0-1 Integer Programming and Network Flow problems.
- When it comes to combinatory optimization problems, branch and bound work well. An optimization problem is optimized by combinatory optimization by finding its maximum or minimum based on its objective function. The combinatory optimization problems include Boolean Satisfiability and Integer Linear Programming.

Basic Concepts of Branch and Bound:

▸ **Generation of a state space tree:**
As in the case of backtracking, B&B generates a state space tree to efficiently search the solution space of a given problem instance.

In B&B, all children of an E-node in a state space tree are produced before any live node gets converted in an E-node. Thus, the E-node remains an E-node until i becomes a dead node.

- **Evaluation of a candidate solution:**

Unlike backtracking, B&B needs additional factors evaluate a candidate solution:

1. A way to assign a bound on the best values of the given criterion functions to each node in a state space tree: It is produced by the addition of further components to the partial solution given by that node.
2. The best values of a given criterion function obtained so far: It describes the upper bound for the maximization problem and the lower bound for the minimization problem.

- A feasible solution is defined by the problem states that satisfy all the given constraints.
- An **optimal solution** is a feasible solution, which produces the best value of a given objective function.
- **Bounding function :**

It optimizes the search for a solution vector in the solution space of a given problem instance.

It is a heuristic function that evaluates the lower and upper bounds on the possible solutions at each node. The bound values are used to search the

partial solutions leading to an optimal solution. If a node does not produce a solution better than the best solution obtained thus far, then it is abandoned without further exploration.

The algorithm then branches to another path to get a better solution. The desired solution to the problem is the value of the best solution produced so far.

▸ The reasons to dismiss a search path at the current node :
(i) The bound value of the node is lower than the upper bound in the case of the maximization problem and higher than the lower bound in the case of the minimization problem. (i.e. the bound value of the ade is not better than the value of the best solution obtained until that node).

(ii) The node represents infeasible solutions, de violation of the constraints of the problem.

(iii) The node represents a subset of a feasible solution containing a single point. In this case, if the latest solution is better than the best solution obtained so far the best solution is modified to the value of a feasible solution at that node.

Types of Branch and Bound Solutions:
The solution of the Branch and the bound problem can be represented in two ways:

- **Variable size solution:** Using this solution, we can find the subset of the given set that gives the optimized solution to the given problem. For example, if we have to select a combination of elements from {A, B, C, D} that optimizes the given problem, and it is found that A and B together give the best solution, then the solution will be {A, B}.
- **Fixed-size solution:** There are 0s and 1s in this solution, with the digit at the ith position indicating whether the ith element should be included, for the above example, the solution will be given by {1, 1, 0, 0}, here 1 represent that we have select the element which at ith position and 0 represent we don’t select the element at ith position.

Lecture 3

Classification of Branch and Bound Problems:
The Branch and Bound method can be classified into three types based on the order in which the state space tree is searched.

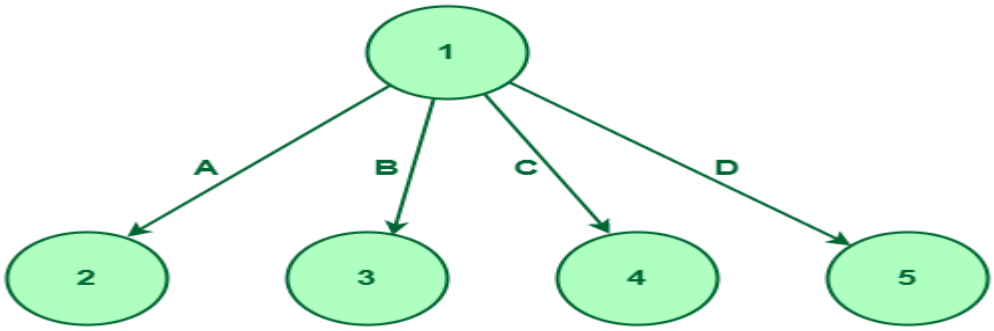
1. FIFO Branch and Bound
2. LIFO Branch and Bound
3. Least Cost-Branch and Bound

We will now discuss each of these methods in more detail. To denote the solutions in these methods, we will use the variable solution method.

1. FIFO Branch and Bound

First-In-First-Out is an approach to the branch and bound problem that uses the queue approach to create a state-space tree. In this case, the breadth-first search is performed, that is, the elements at a certain level are all searched, and then the elements at the next level are searched, starting with the first child of the first node at the previous level.

For a given set {A, B, C, D}, the state space tree will be constructed as follows :

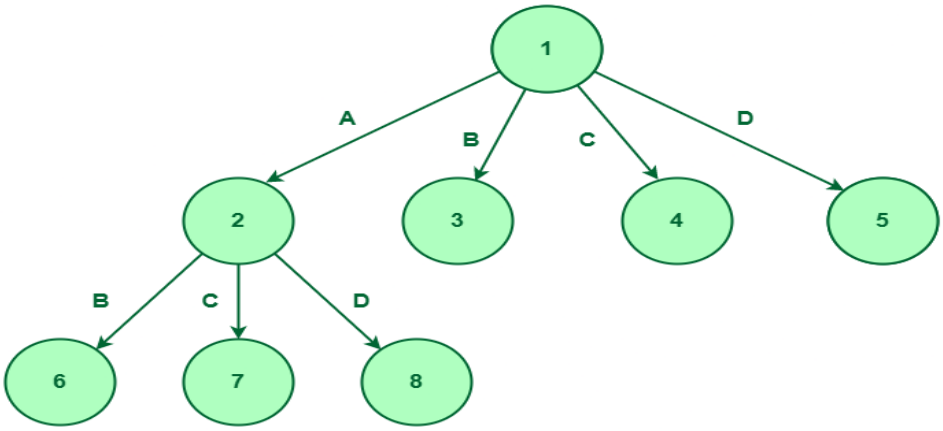


State Space tree for set {A, B, C, D}

The above diagram shows that we first consider element A, then element B, then element C and finally we'll consider the last element which is D. We are performing BFS while exploring the nodes.

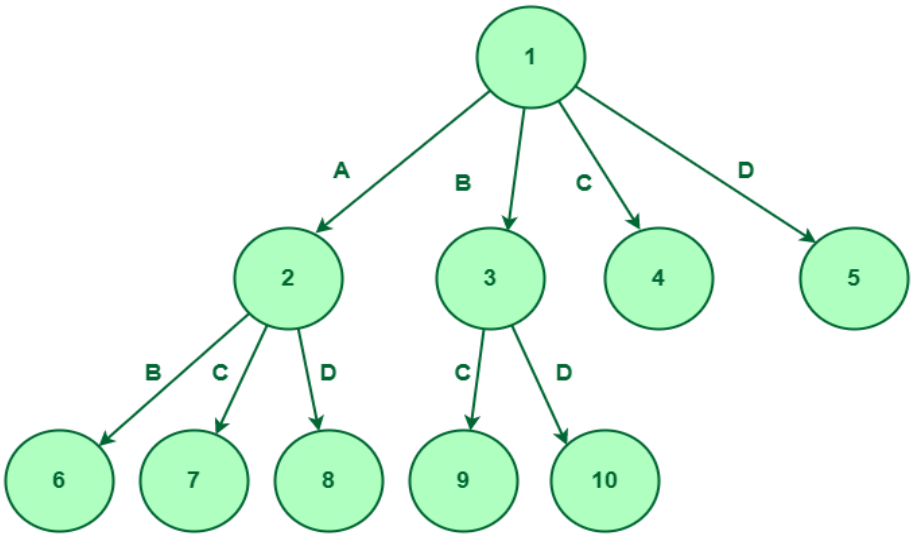
So, once the first level is completed. We'll consider the first element, then we can consider either B, C, or D. If we follow the route then it says that we are doing elements A and D so we will not consider elements B

and C. If we select the elements A and D only, then it says that we are selecting elements A and D and we are not considering elements B and C.



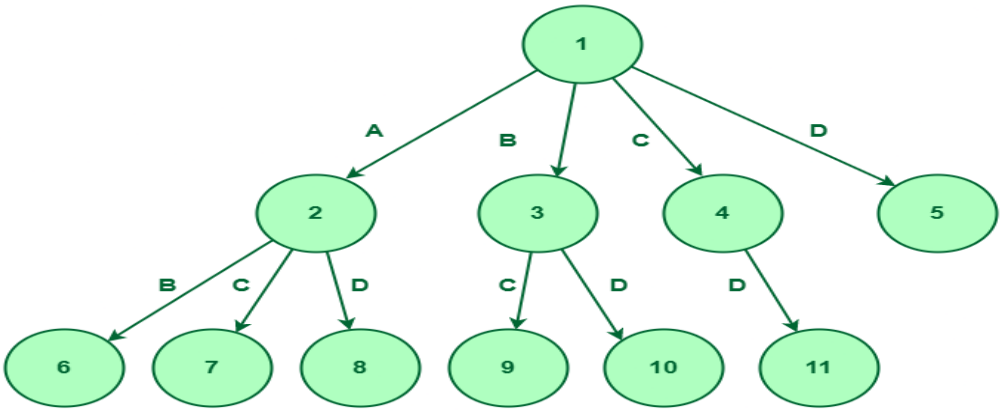
Selecting element A

Now, we will expand node 3, as we have considered element B and not considered element A, so, we have two options to explore that is elements C and D. Let's create nodes 9 and 10 for elements C and D respectively.



Considered element B and not considered element A

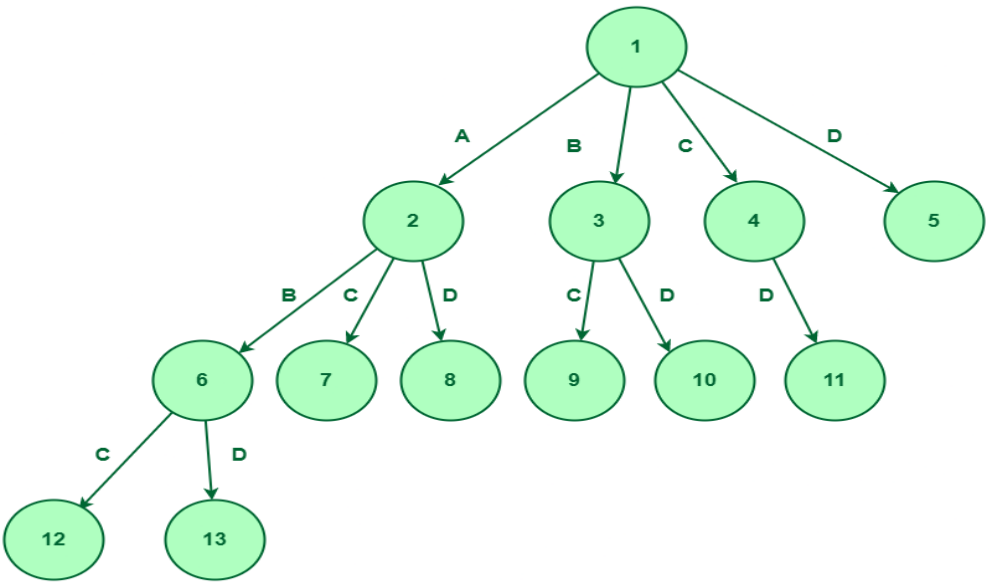
Now, we will expand node 4 as we have only considered elements C and not considered elements A and B, so, we have only one option to explore which is element D. Let's create node 11 for D.



Considered elements C and not considered elements A and B

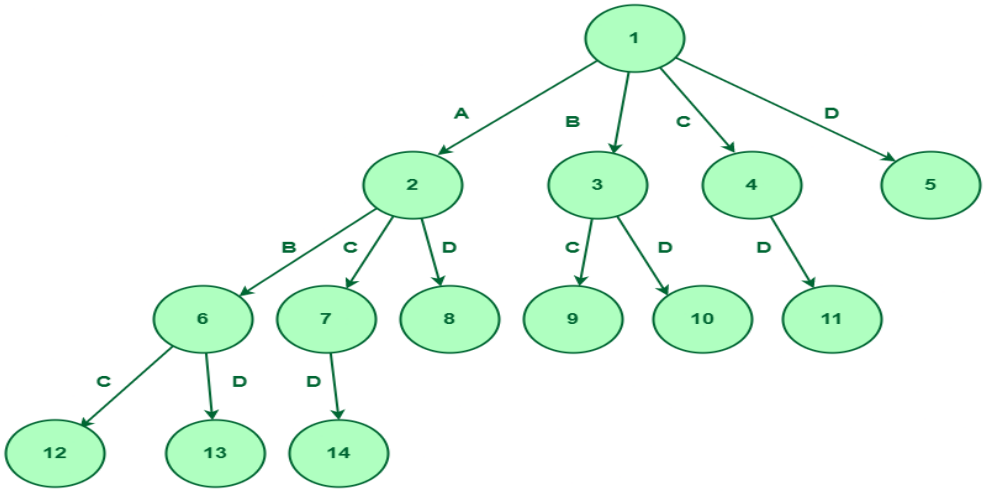
Till node 5, we have only considered elements D, and not selected elements A, B, and C. So, We have no more elements to explore, Therefore on node 5, there won't be any expansion.

Now, we will expand node 6 as we have considered elements A and B, so, we have only two option to explore that is element C and D. Let's create node 12 and 13 for C and D respectively.



Expand node 6

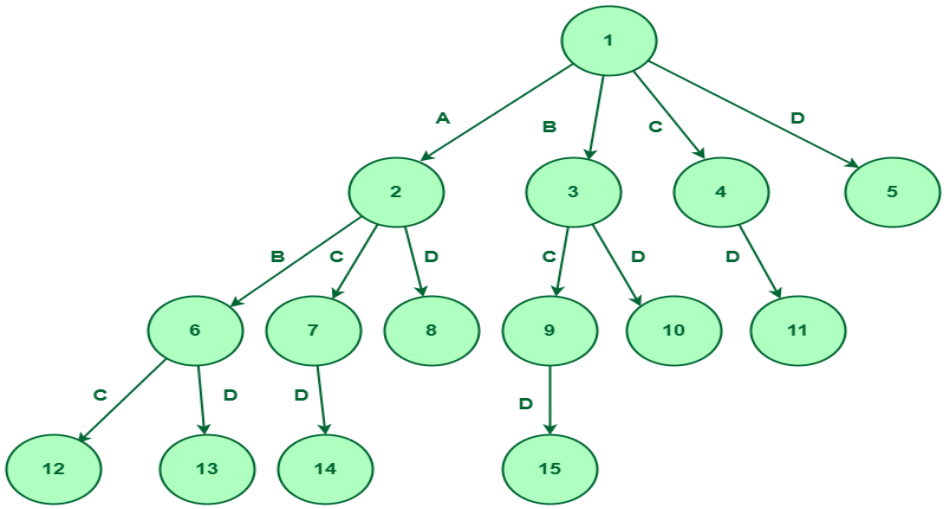
Now, we will expand node 7 as we have considered elements A and C and not consider element B, so, we have only one option to explore which is element D. Let's create node 14 for D.



Expand node 7

Till node 8, we have considered elements A and D, and not selected elements B and C, So, We have no more elements to explore, Therefore on node 8, there won't be any expansion.

Now, we will expand node 9 as we have considered elements B and C and not considered element A, so, we have only one option to explore which is element D. Let's create node 15 for D.

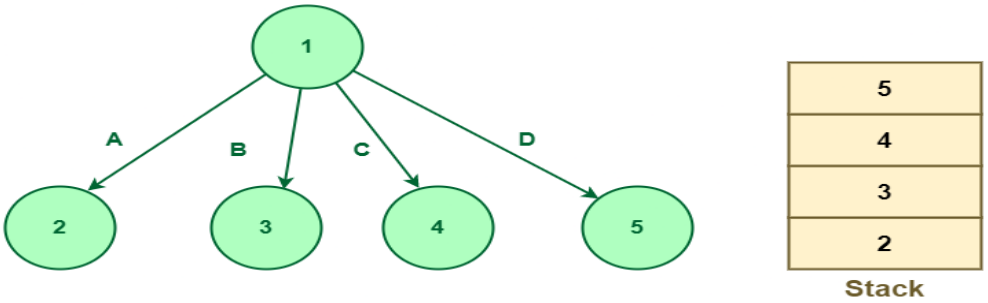


Expand node 9

2. LIFO Branch and Bound

The Last-In-First-Out approach for this problem uses stack in creating the state space tree. When nodes are added to a state space tree, they are added to a stack. After all nodes of a level have been added, we pop the topmost element from the stack and explore it.

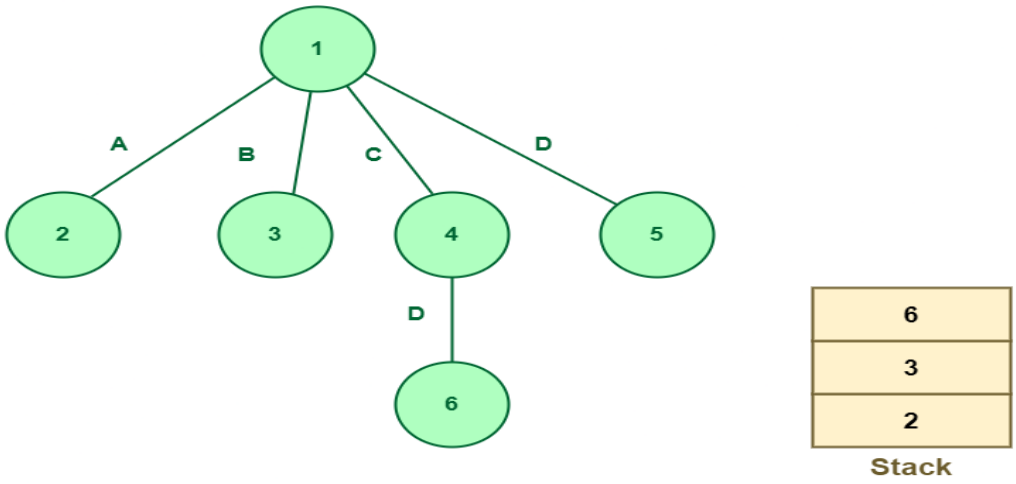
For a given set {A, B, C, D}, the state space tree will be constructed as follows :



State space tree for element {A, B, C, D}

Now the expansion would be based on the node that appears on the top of the stack. Since node 5 appears on the top of the stack, so we will expand node 5. We will pop out node 5 from the stack. Since node 5 is in the last element, i.e., D so there is no further scope for expansion.

The next node that appears on the top of the stack is node 4. Pop-out node 4 and expand. On expansion, element D will be considered and node 6 will be added to the stack shown below:

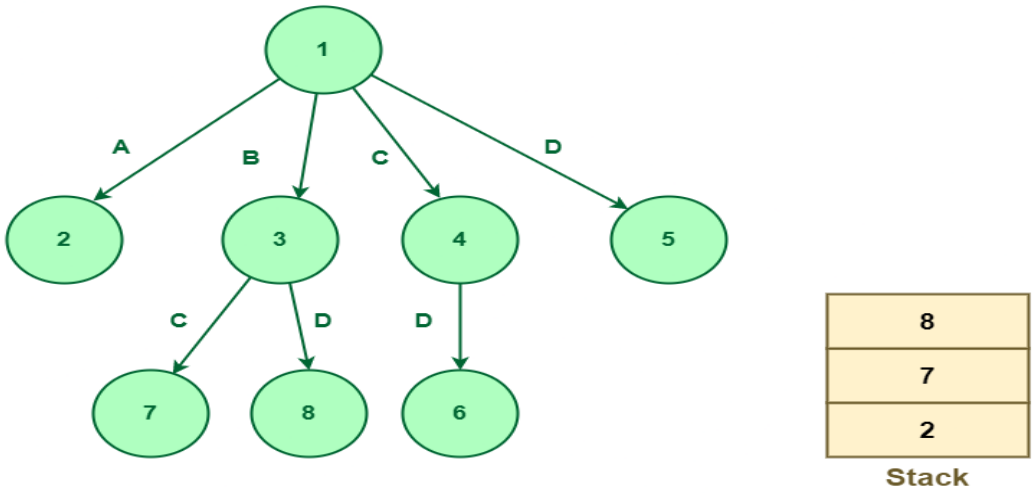


Expand node 4

The next node is 6 which is to be expanded. Pop-out node 6 and expand. Since node 6 is in the last element, i.e., D so there is no further scope for expansion.

The next node to be expanded is node 3. Since node 3 works on element B so node 3 will be expanded to two nodes, i.e., 7 and 8 working on elements C and D respectively. Nodes 7 and 8 will be pushed into the stack.

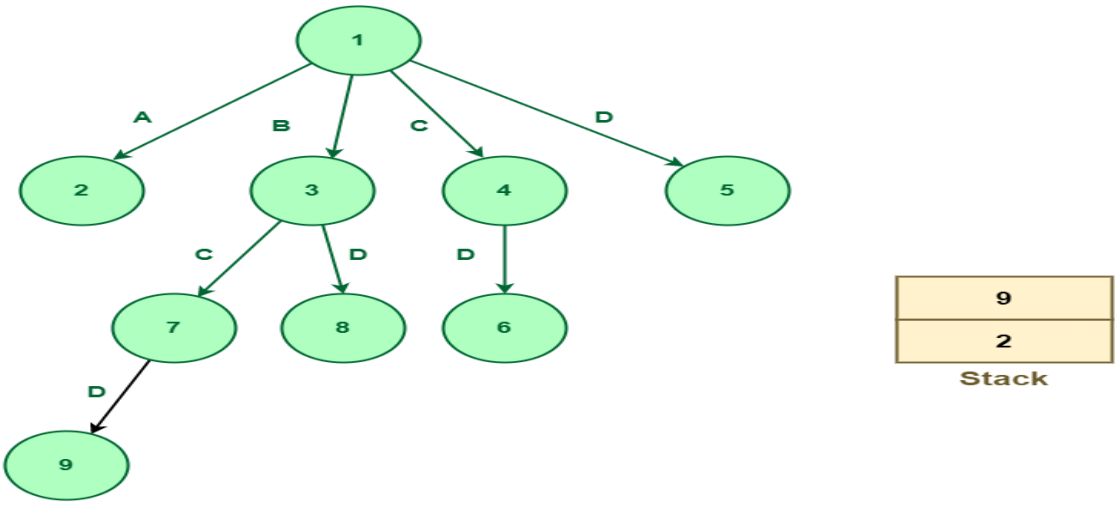
The next node that appears on the top of the stack is node 8. Pop-out node 8 and expand. Since node 8 works on element D so there is no further scope for the expansion.



Expand node 3

The next node that appears on the top of the stack is node 7. Pop-out node 7 and expand. Since node 7 works on element C so node 7 will be further expanded to node 9 which works on element D and node 9 will be pushed into the stack.

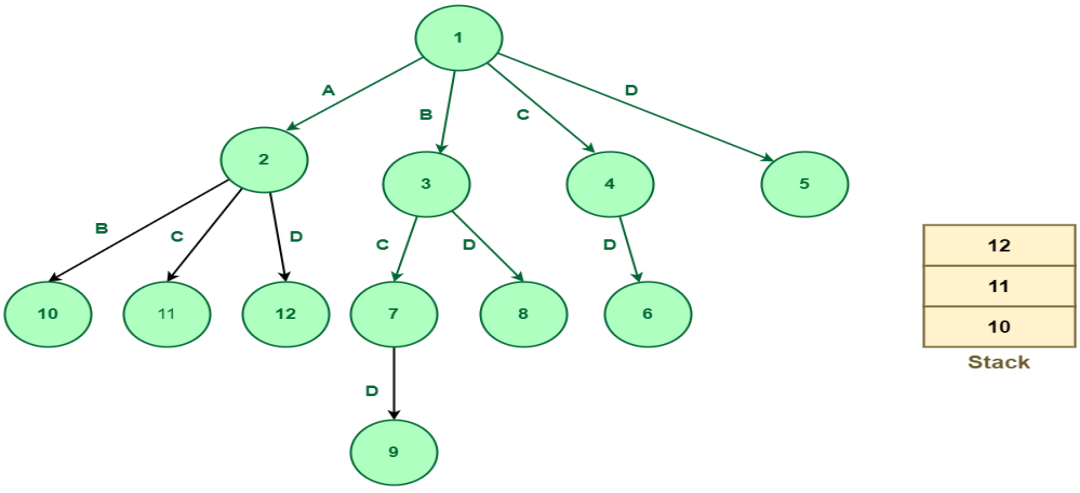
The next node is 6 which is to be expanded. Pop-out node 6 and expand. Since node 6 is in the last element, i.e., D so there is no further scope for expansion.



Expand node 7

The next node that appears on the top of the stack is node 9. Since node 9 works on element D, there is no further scope for expansion.

The next node that appears on the top of the stack is node 2. Since node 2 works on the element A so it means that node 2 can be further expanded. It can be expanded up to three nodes named 10, 11, 12 working on elements B, C, and D respectively. There new nodes will be pushed into the stack shown as below:



Expand node 2

In the above method, we explored all the nodes using the stack that follows the LIFO principle.

Lecture 4

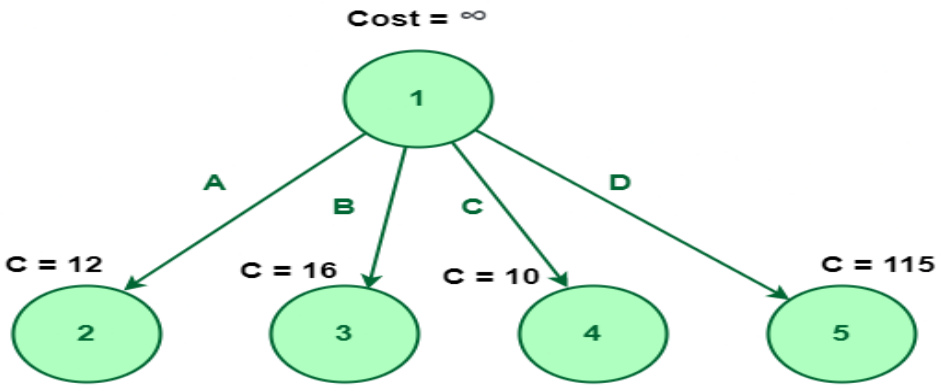
Least Cost-Branch and Bound

To explore the state space tree, this method uses the cost function. The previous two methods also calculate the cost function at each node but the cost is not been used for further exploration.

In this technique, nodes are explored based on their costs, the cost of the node can be defined using the problem and with the help of the given problem, we can define the cost function. Once the cost function is defined, we can define the cost of the node.
Now, Consider a node whose cost has been determined. If this value is greater than U0, this node or its children will not be able to give a solution. As a result, we can kill this node and not explore its further branches. As a result, this method prevents us from exploring cases that are not worth it, which makes it more efficient for us.

Let's first consider node 1 having cost infinity shown below:

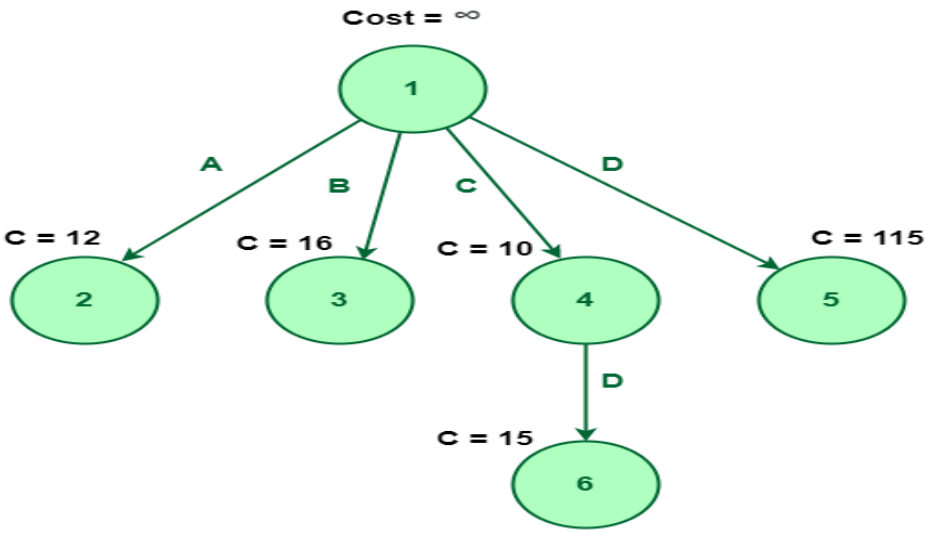
In the following diagram, node 1 is expanded into four nodes named 2, 3, 4, and 5.



Node 1 is expanded into four nodes named 2, 3, 4, and 5

Assume that cost of the nodes 2, 3, 4, and 5 are 12, 16, 10, and 315 respectively.
In this method, we will explore the node which is having the least cost. In the above figure, we can observe that the node with a minimum cost is node 4. So, we will explore node 4 having a cost of 10.

During exploring node 4 which is element C, we can notice that there is only one possible element that remains unexplored which is D (i.e, we already decided not to select elements A, and B). So, it will get expanded to one single element D, let's say this node number is 6.



Exploring node 4 which is element C

Now, Node 6 has no element left to explore. So, there is no further scope for expansion. Hence the element {C, D} is the optimal way to choose for the least cost.

Lecture 5

APPLICATIONS:

This approach is used for a number of NP-hard problems:

- Integer programming
- Nonlinear programming
- Travelling salesman problem (TSP)
- Quadratic assignment problem (QAP)
- Maximum satisfiability problem (MAX-SAT)
- Nearest neighbor search (by Keinosuke Fukunaga)
- Flow shop scheduling
- Cutting stock problem
- Computational phylogenetics
- Set inversion
- Parameter estimation
- 0/1 knapsack problem
- Set cover problem
- Feature selection in machine learning
- Structured prediction in computer vision
- Arc routing problem, including Chinese Postman problem
- Talent Scheduling, scenes shooting arrangement problem

Branch-and-bound may also be a base of various heuristics. For example, one may wish to stop branching when the gap between the upper and lower bounds becomes smaller than a certain threshold. This is used when the solution is "good enough for practical purposes" and can greatly reduce the computations required. This type of solution is particularly applicable when the cost function used is *noisy* or is the result of statistical estimates and so is not known precisely but rather only known to lie within a range of values with a specific probability.

Lecture 6

0/1 Knapsack Problem

Prerequisites: Introduction to Knapsack Problem, its Types and How to solve them

Given **N** items where each item has some weight and profit associated with it and also given a bag with capacity **W**, [i.e., the bag can hold at most **W** weight in it]. The task is to put the items into the bag such that the sum of profits associated with them is the maximum possible.

Note: The constraint here is we can either put an item completely into the bag or cannot put it at all [It is not possible to put a part of an item into the bag].

Examples:

Input: N = 3, W = 4, profit[] = {1, 2, 3}, weight[] = {4, 5, 1}
Output: 3

Explanation: There are two items which have weight less than or equal to 4. If we select the item with weight 4, the possible profit is 1. And if we select the item with weight 1, the possible profit is 3. So the maximum possible profit is 3. Note that we cannot put both the items with weight 4 and 1 together as the capacity of the bag is 4.

Input: N = 3, W = 3, profit[] = {1, 2, 3}, weight[] = {4, 5, 6}
Output: 0

0 - 1 Knapsack Problem

Recursion Approach for 0/1 Knapsack Problem:

To solve the problem follow the below idea:

A simple solution is to consider all subsets of items and calculate the total weight and profit of all subsets. Consider the only subsets whose total weight is smaller than W. From all such subsets, pick the subset with maximum profit.

Optimal Substructure: To consider all subsets of items, there can be two cases for every item.

- **Case 1:** The item is included in the optimal subset.
- **Case 2:** The item is not included in the optimal set.

Follow the below steps to solve the problem:

The maximum value obtained from 'N' items is the max of the following two values.

- Case 1 (include the Nth item): Value of the Nth item plus maximum value obtained by remaining N-1 items and remaining weight i.e. (W-weight of the Nth item).
- Case 2 (exclude the Nth item): Maximum value obtained by N-1 items and W weight.
- If the weight of the 'Nth' item is greater than 'W', then the Nth item cannot be included and **Case 2** is the only possibility.

Time Complexity: O(2N)

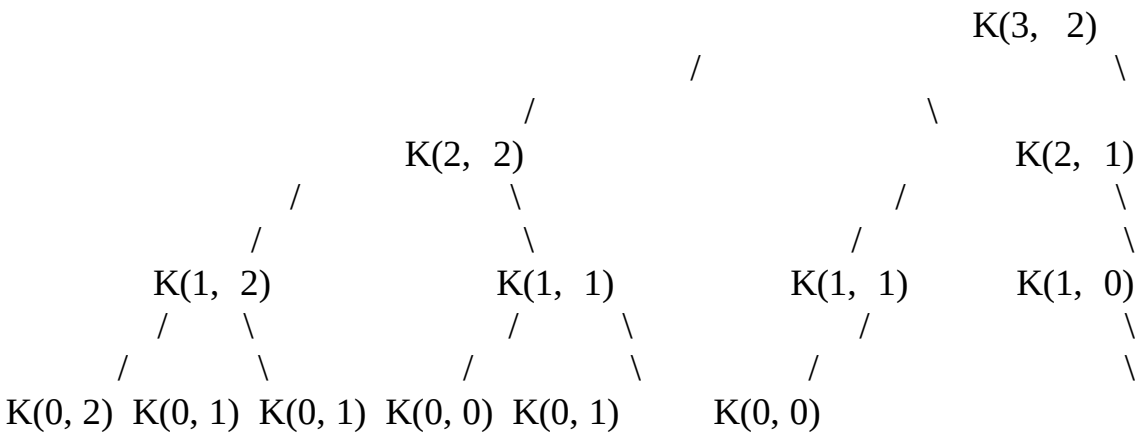
Auxiliary Space: O(N), Stack space required for recursion

Dynamic Programming Approach for 0/1 Knapsack Problem

Memoization Approach for 0/1 Knapsack Problem:

Note: It should be noted that the above function using recursion computes the same subproblems again and again. See the following recursion tree, K(1, 1) is being evaluated twice.

In the following recursion tree, **K()** refers to knapSack(). The two parameters indicated in the following recursion tree are n and W. The recursion tree is for following sample inputs. weight[] = {1, 1, 1}, W = 2, profit[] = {10, 20, 30}



Recursion tree for Knapsack capacity 2 units and 3 items of 1 unit weight.

As there are repetitions of the same subproblem again and again we can implement the following idea to solve the problem.

If we get a subproblem the first time, we can solve this problem by creating a 2-D array that can store a particular state (n, w). Now if we come across the same state (n, w) again instead of calculating it in exponential complexity we can directly return its result stored in the table in constant time.

Time Complexity: O(N * W). As redundant calculations of states are avoided.

Auxiliary Space: O(N * W) + O(N). The use of a 2D array data structure for storing intermediate states and O(N) auxiliary stack space(ASS) has been used for recursion stack

Bottom-up Approach for 0/1 Knapsack Problem:

To solve the problem follow the below idea:
Since subproblems are evaluated again, this problem has Overlapping Sub-problems property. So the 0/1 Knapsack problem has both properties of a dynamic programming problem. Like other typical Dynamic Programming(DP) problems, re-computation of the same subproblems can be avoided by constructing a temporary array K[][] in a bottom-up manner.

Illustration:

Below is the illustration of the above approach:
Let, **weight[] = {1, 2, 3}, profit[] = {10, 15, 40}, Capacity = 6**

- If no element is filled, then the possible profit is 0.

weight⇢ item⇣/	**0**	**1**	**2**	**3**	**4**	**5**	**6**
0	0	0	0	0	0	0	0
1							
2							
3							

- **For filling the first item in the bag:** If we follow the above mentioned procedure, the table will look like the following.

weight⇢ item⇣/	**0**	**1**	**2**	**3**	**4**	**5**	**6**
0	0	0	0	0	0	0	0
1	0	10	10	10	10	10	10
2							
3							

- **For filling the second item:**
  When jthWeight = 2, then maximum possible profit is max (10, DP[1][2-2] + 15) = max(10, 15) = 15.
  When jthWeight = 3, then maximum possible profit is max(2 not put, 2 is put into bag) = max(DP[1][3], 15+DP[1][3-2]) = max(10, 25) = 25.

weight⇢ item⇣/	**0**	**1**	**2**	**3**	**4**	**5**	**6**
0	0	0	0	0	0	0	0
1	0	10	10	10	10	10	10
2	0	10	15	25	25	25	25
3							

- **For filling the third item:**
  When jthWeight = 3, the maximum possible profit is max(DP[2][3], 40+DP[2][3-3]) = max(25, 40) = 40.
  When jthWeight = 4, the maximum possible profit is max(DP[2][4], 40+DP[2][4-3]) = max(25, 50) = 50.
  When jthWeight = 5, the maximum possible profit is max(DP[2][5], 40+DP[2][5-3]) = max(25, 55) = 55.
  When jthWeight = 6, the maximum possible profit is max(DP[2][6], 40+DP[2][6-3]) = max(25, 65) = 65.

weight⇢ item⇣/	**0**	**1**	**2**	**3**	**4**	**5**	**6**
0	0	0	0	0	0	0	0
1	0	10	10	10	10	10	10
2	0	10	15	25	25	25	25
3	0	10	15	40	50	55	65

Below is the implementation of the above approach: